

# RANDOMIZED AUTOMATIC DIFFERENTIATION

Deniz Oktay,<sup>1</sup> Nick McGreivy,<sup>2</sup> Joshua Aduol,<sup>1</sup> Alex Beatson,<sup>1</sup> Ryan P. Adams<sup>1</sup>

Princeton University

Princeton, NJ

{doktay, mcgreivy, jaduol, abeatson, rpa}@princeton.edu

## ABSTRACT

The successes of deep learning, variational inference, and many other fields have been aided by specialized implementations of reverse-mode automatic differentiation (AD) to compute gradients of mega-dimensional objectives. The AD techniques underlying these tools were designed to compute exact gradients to numerical precision, but modern machine learning models are almost always trained with stochastic gradient descent. Why spend computation and memory on exact (minibatch) gradients only to use them for stochastic optimization? We develop a general framework and approach for randomized automatic differentiation (RAD), which can allow unbiased gradient estimates to be computed with reduced memory in return for variance. We examine limitations of the general approach, and argue that we must leverage problem specific structure to realize benefits. We develop RAD techniques for a variety of simple neural network architectures, and show that for a fixed memory budget, RAD converges in fewer iterations than using a small batch size for feedforward networks, and in a similar number for recurrent networks. We also show that RAD can be applied to scientific computing, and use it to develop a low-memory stochastic gradient method for optimizing the control parameters of a linear reaction-diffusion PDE representing a fission reactor.

## 1 INTRODUCTION

Deep neural networks have taken center stage as a powerful way to construct and train massively-parametric machine learning (ML) models for supervised, unsupervised, and reinforcement learning tasks. There are many reasons for the resurgence of neural networks—large data sets, GPU numerical computing, technical insights into overparameterization, and more—but one major factor has been the development of tools for automatic differentiation (AD) of deep architectures. Tools like PyTorch and TensorFlow provide a computational substrate for rapidly exploring a wide variety of differentiable architectures without performing tedious and error-prone gradient derivations. The flexibility of these tools has enabled a revolution in AI research, but the underlying ideas for reverse-mode AD go back decades. While tools like PyTorch and TensorFlow have received huge dividends from a half-century of AD research, they are also burdened by the baggage of design decisions made in a different computational landscape. The research on AD that led to these ubiquitous deep learning frameworks is focused on the computation of Jacobians that are *exact* up to numerical precision. However, in modern workflows these Jacobians are used for *stochastic* optimization. We ask:

*Why spend resources on exact gradients when we’re going to use stochastic optimization?*

This question is motivated by the surprising realization over the past decade that deep neural network training can be performed almost entirely with first-order stochastic optimization. In fact, empirical evidence supports the hypothesis that the regularizing effect of gradient noise *assists* model generalization (Keskar et al., 2017; Smith & Le, 2018; Hochreiter & Schmidhuber, 1997). Stochastic gradient descent variants such as AdaGrad (Duchi et al., 2011) and Adam (Kingma & Ba, 2015) form the core of almost all successful optimization techniques for these models, using small subsets of the data to form the noisy gradient estimates.

<sup>1</sup>Department of Computer Science

<sup>2</sup>Department of Astrophysical Sciences

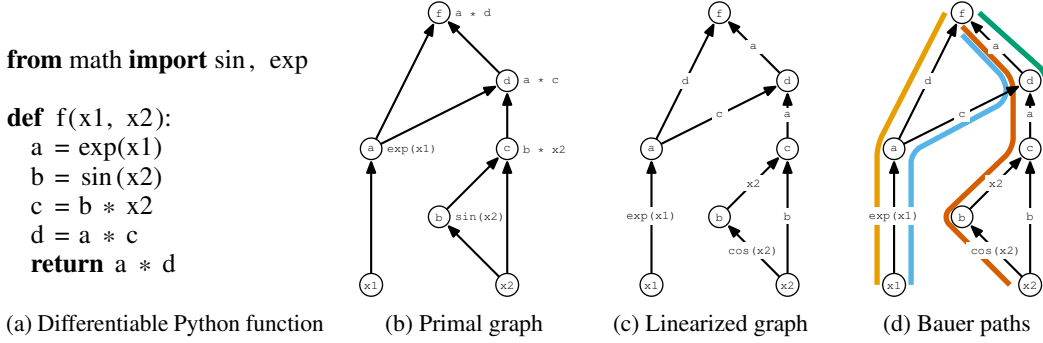


Figure 1: Illustration of the basic concepts of the linearized computational graph and Bauer’s formula. (a) a simple Python function with intermediate variables; (b) the primal computational graph, a DAG with variables as vertices and flow moving upwards to the output; (c) the linearized computational graph (LCG) in which the edges are labeled with the values of the local derivatives; (d) illustration of the four paths that must be evaluated to compute the Jacobian. (Example from Paul D. Hovland.)

The goals and assumptions of automatic differentiation as performed in classical and modern systems are mismatched with those required by stochastic optimization. Traditional AD computes the derivative or Jacobian of a function accurately to numerical precision. This accuracy is required for many problems in applied mathematics which AD has served, e.g., solving systems of differential equations. But in stochastic optimization we can make do with inaccurate gradients, as long as our estimator is unbiased and has reasonable variance. We ask the same question that motivates mini-batch SGD: why compute an exact gradient if we can get noisy estimates cheaply? By thinking of this question in the context of AD, we can go beyond mini-batch SGD to more general schemes for developing cheap gradient estimators: in this paper, we focus on developing gradient estimators with low memory cost. Although previous research has investigated approximations in the forward or reverse pass of neural networks to reduce computational requirements, here we replace deterministic AD with *randomized* automatic differentiation (RAD), trading off of computation for variance *inside* AD routines when imprecise gradient estimates are tolerable, while retaining unbiasedness.

## 2 AUTOMATIC DIFFERENTIATION

Automatic (or *algorithmic*) differentiation is a family of techniques for taking a program that computes a differentiable function  $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ , and producing another program that computes the associated derivatives; most often the Jacobian:  $\mathcal{J}[f] = f' : \mathbb{R}^n \rightarrow \mathbb{R}^{m \times n}$ . (For a comprehensive treatment of AD, see Griewank & Walther (2008); for an ML-focused review see Baydin et al. (2018).) In most machine learning applications,  $f$  is a loss function that produces a scalar output, i.e.,  $m = 1$ , for which the gradient with respect to parameters is desired. AD techniques are contrasted with the method of *finite differences*, which approximates derivatives numerically using a small but non-zero step size, and also distinguished from *symbolic differentiation* in which a mathematical expression is processed using standard rules to produce another mathematical expression, although Elliott (2018) argues that the distinction is simply whether or not it is the compiler that manipulates the symbols.

There are a variety of approaches to AD: source-code transformation (e.g., Bischof et al. (1992); Hascoet & Pascual (2013); van Merriënboer et al. (2018)), execution tracing (e.g., Walther & Griewank (2009); Maclaurin et al.), manipulation of explicit computational graphs (e.g., Abadi et al. (2016); Bergstra et al. (2010)), and category-theoretic transformations (Elliott, 2018). AD implementations exist for many different host languages, although they vary in the extent to which they take advantage of native programming patterns, control flow, and language features. Regardless of whether it is constructed at compile-time, run-time, or via an embedded domain-specific language, all AD approaches can be understood as manipulating the *linearized computational graph* (LCG) to collapse out intermediate variables. Figure 1 shows the LCG for a simple example. These computational graphs are always directed acyclic graphs (DAGs) with vertices as variables.

Let the outputs of  $f$  be  $y_j$ , the inputs  $\theta_i$ , and the intermediates  $z_l$ . AD can be framed as the computation of a partial derivative as a sum over all paths through the LCG DAG (Bauer, 1974):

$$\frac{\partial y_j}{\partial \theta_i} = \mathcal{J}_\theta[f]_{j,i} = \sum_{[i \rightarrow j]} \prod_{(k,l) \in [i \rightarrow j]} \frac{\partial z_l}{\partial z_k} \quad (1)$$

where  $[i \rightarrow j]$  indexes paths from vertex  $i$  to vertex  $j$  and  $(k, l) \in [i \rightarrow j]$  denotes the set of edges in that path. See Figure 1d for an illustration. Although general, this naïve sum over paths does not take advantage of the structure of the problem and so, as in other kinds of graph computations, dynamic programming (DP) provides a better approach. DP collapses substructures of the graph until it becomes bipartite and the remaining edges from inputs to outputs represent exactly the entries of the Jacobian matrix. This is referred to as the *Jacobian accumulation problem* (Naumann, 2004) and there are a variety of ways to manipulate the graph, including vertex, edge, and face elimination (Griewank & Naumann, 2002). Forward-mode AD and reverse-mode AD (backpropagation) are special cases of more general dynamic programming strategies to perform this summation; determination of the optimal accumulation schedule is unfortunately NP-complete (Naumann, 2008).

While the above formulation in which each variable is a scalar can represent any computational graph, it can lead to structures that are difficult to reason about. Often we prefer to manipulate vectors and matrices, and we can instead let each intermediate  $z_l$  represent a  $d_l$  dimensional vector. In this case,  $\partial z_l / \partial z_k \in \mathbb{R}^{d_l \times d_k}$  represents the intermediate Jacobian of the operation  $z_k \rightarrow z_l$ . Note that Equation 1 now expresses the Jacobian of  $f$  as a sum over chained matrix products.

### 3 RANDOMIZING AUTOMATIC DIFFERENTIATION

We introduce techniques that could be used to decrease the resource requirements of AD when used for stochastic optimization. We focus on functions with a scalar output where we are interested in the gradient of the output with respect to some parameters,  $\mathcal{J}_\theta[f]$ . Reverse-mode AD efficiently calculates  $\mathcal{J}_\theta[f]$ , but requires the full linearized computational graph to either be stored during the forward pass, or to be recomputed during the backward pass using intermediate variables recorded during the forward pass. For large computational graphs this could provide a large memory burden.

The most common technique for reducing the memory requirements of AD is gradient checkpointing (Griewank & Walther, 2000; Chen et al., 2016), which saves memory by adding extra forward pass computations. Checkpointing is effective when the number of "layers" in a computation graph is much larger than the memory required at each layer. We take a different approach; we instead aim to save memory by increasing gradient variance, without extra forward computation.

Our main idea is to consider an unbiased estimator  $\hat{\mathcal{J}}_\theta[f]$  such that  $\mathbb{E}\hat{\mathcal{J}}_\theta[f] = \mathcal{J}_\theta[f]$  which allows us to save memory required for reverse-mode AD. Our approach is to determine a sparse (but random) linearized computational graph during the forward pass such that reverse-mode AD applied on the sparse graph yields an unbiased estimate of the true gradient. Note that the original computational graph is used for the forward pass, and randomization is used to determine a LCG to use for the backward pass in place of the original computation graph. We may then decrease memory costs by storing the sparse LCG directly or storing intermediate variables required to compute the sparse LCG.

In this section we provide general recipes for randomizing AD by sparsifying the LCG. In sections 4 and 5 we apply these recipes to develop specific algorithms for neural networks and linear PDEs which achieve concrete memory savings.

#### 3.1 PATH SAMPLING

Observe that in Bauer’s formula each Jacobian entry is expressed as a sum over paths in the LCG. A simple strategy is to sample paths uniformly at random from the computation graph, and form a Monte Carlo estimate of Equation 1. Naïvely this could take multiple passes through the graph. However, multiple paths can be sampled without significant computation overhead by performing a topological sort of the vertices and iterating through vertices, sampling multiple outgoing edges for each. We provide a proof and detailed algorithm in the appendix. Dynamic programming methods such as reverse-mode automatic differentiation can then be applied to the sparsified LCG.